

Week 2 Tutorial: Arithmetic, Libraries, Strings, Lists, and Tuples

This tutorial supports both Lecture 3 and Lecture 4. It has three parts:

1. You prepare for Lecture 3 at home with very basic exercises in Part 1.
2. I explain difficult concepts used in Lecture 3 in Part 2.
3. I prepare you for the upcoming lecture 4 on lists, strings, and tuples in Part 3.

The central teaching cycle is **explain, predict, run, modify**. You should predict the result before running code and use errors as evidence about how Python works.

ILO

- **KU2:** Choose appropriate data types and data structures for different kinds of data, depending on their performance characteristics.
- **KU3:** Design algorithms to solve simple programming problems.
- **CS1:** Structure small programs by the use of iterations, functions, modules, classes, and methods.
- **CS3:** Form readable, descriptive and well-documented program code.
- **CS5:** Express mathematical formulas as programming language expressions and algorithms.
- **CS8:** Use programming tools such as text editor, command line interface, and IDE.

Objectives

After completing this tutorial, you should be able to:

1. Use arithmetic operators and explain the difference between `/`, `//`, and `%`.
2. Use an accumulator and a loop to process values.
3. Explain why floating-point values are approximations.
4. Format floating-point output to a fixed number of decimal places.
5. Explain what a Python library is and import functions from a library.
6. Store several values in a list and change the list when needed.
7. Sort a list, access individual elements, and take a slice of a list.
8. Use string methods and split a string into a list.
9. Use a tuple when a fixed group of values belongs together.

Part 1: Preparation for Lecture 3

This part explains basic arithmetic ideas that you may find difficult in Lecture 3. You should do these exercises at home before coming to the lecture.

Home preparation A: Arithmetic operators

See the following code. First, try to predict the output of these print statements. Then type them in a code file and run the program.

```
print(10 / 3)
print(10 // 3)
print(10 % 3)
print(2 ** 4)
```

Compare your prediction with the program output.

A short explanation of these operators is as follows:

- `/` performs floating-point division;
- `//` performs floor division;
- `%` gives the remainder;
- `**` raises a number to a power.

Home preparation B: Floating-point values

Predict the output of the following code before writing it as a program:

```
print(0.1 + 0.2)
print(1 / 3)
print(f"{1 / 3:.2f}") # display only two decimal places
```

A float may contain more digits than you expect. Python may also display more digits than you need. The f-string in the last statement tells Python to display the result with two digits after the decimal point.

Home preparation C: A short accumulator

Trace the value of `total` after every loop iteration:

```
total = 0

for number in range(1, 4):
    total = total + number

print(total)
```

Then change the program to calculate the sum of the numbers from 1 to 5.

Home preparation D: Order of operations

Predict and run both statements (add parentheses to make the intended calculation clear):

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

Part 2: Libraries and Lecture 3

In this part, I explain what libraries are and why we use them. A library is a collection of code that has already been written for us. We import a library when we need functionality that Python does not provide directly, or when using an existing solution is clearer and more reliable than writing one ourselves.

For example, the `random` library provides functions for generating random values, and the `math` library provides mathematical constants and functions.

Activity 1: Importing and using a library

First, use a function from the `random` library:

```
import random

point = random.randint(0, 100)
print(point)
```

Run the program several times. The output may be different each time because `randint(0, 100)` returns a random integer between 0 and 100, including both limits.

Then import only the function that you need:

```
from random import randint

point = randint(0, 100)
print(point)
```

Discuss the difference between `random.randint(...)` and `randint(...)`.

Activity 2: Generate points for five students

Generate a random point value (between 50 to 100) for each of five students:

```
from random import randint

for student in range(5):
    points = randint(50, 100)
    print("Student", student + 1, "got", points, "points")
```

Now put the generation into a function:

```
from random import randint
```

```
def generate_points(number_of_students):  
    for student in range(number_of_students):  
        points = randint(50, 100)  
        print("Student", student + 1, "got", points, "points")  
  
generate_points(5)
```

The function makes the program reusable: we can generate points for 5, 10, or 100 students without rewriting the loop.

Activity 3: The `math` library

The `math` library contains constants and functions used in mathematical calculations:

```
import math  
  
print(math.pi)  
print(math.sqrt(25))  
print(math.isclose(0.1 + 0.2, 0.3))
```

Explain the purpose of each expression:

- `math.pi` gives an approximation of pi;
- `math.sqrt(25)` calculates a square root;
- `math.isclose(...)` checks whether two floating-point values are close enough to be treated as equal.

This is useful in Lecture 3 when the program calculates areas using `math.pi` and compares floating-point areas.

Part 3: preparation for Lecture 4

In this part, I introduce lists, sorting, indexing, slicing, strings, and tuples. We start with the student-points program from Part 2.

Activity 1: We need to remember the points

In the previous program, each point value disappeared after it was printed. If we want to calculate an average, find the highest point, or compare students later, we need to store all the values.

A list gives us a way to store several values:

```
from random import randint  
  
points = []  
  
for student in range(5):  
    point = randint(0, 100)  
    points.append(point)
```

```
print(points)
```

The list starts empty. The `.append()` method adds one point at a time.

Activity 2: We need to sort the points

If we want to see the points from lowest to highest, we can sort the list:

```
points.sort()  
print(points)
```

The `.sort()` method changes the existing list. This is called an in-place mutation.

Compare it with `sorted()`:

```
sorted_points = sorted(points)  
print(sorted_points)
```

The `sorted()` function creates and returns a sorted result. The list method `.sort()` changes the original list.

Activity 3: Indexing and Slicing

After sorting, we may need the lowest point, the highest point, or the three highest points:

```
lowest = points[0]  
highest = points[-1]  
top_three = points[-3:]  
  
print("Lowest:", lowest)  
print("Highest:", highest)  
print("Top three:", top_three)
```

Explain the difference:

- `points[0]` selects one element;
- `points[-1]` selects the last element;
- `points[-3:]` creates a slice containing the last three elements.

Activity 4: We need student names

Numbers alone are not enough. We need names to identify the students. A string stores text:

```
name = "Ada Lovelace"  
print(name)
```

```
print(name.upper())
print(name[0])
print(name[:3])
```

The string methods and slices create new strings. They do not change the original `name` string.

If names are entered on one line, `.split()` can turn them into a list:

```
names = "Ada Lin Sam Noor Kim".split()
print(names)
```

Now we can access a name with indexing and a group of names with slicing:

```
print(names[0])
print(names[1:4])
```

Activity 5: We need to keep a student record together

A student's name and point value belong together. A tuple can store this fixed pair:

```
student = ("Ada", 87)
print(student)
print(student[0])
print(student[1])
```

We can create a list of student records:

```
students = [("Ada", 87), ("Lin", 92), ("Sam", 76)]
print(students)
```

The list can change when we add or remove students, while each tuple keeps one student's name and points together.